Synthetic Non-Stationary Spiking Data Generation: Dale-Law-Constrained Recurrent Networks with a Switching Poisson GLM

Jan Balewski, NERSC

June 16, 2026

Abstract

We describe the two-script pipeline that produces synthetic non-stationary neural population spike trains for benchmarking latent-state inference algorithms (e.g. PRISM-EM). **Stage 1** constructs a source-column Dale-style weight matrix A_{true} with prescribed spectral radius $R < 1$ and M bias vectors $\{B_m\}_{m=0}^{M-1}$, one per dynamical state. Stationary spike trains under each bias are generated as a firing-rate sanity check. **Stage 2** loads the ground-truth dictionary, schedules transitions among the M states, tracks a smoothly evolving simplex coefficient c_t, and generates the non-stationary spike train from a Poisson GLM with time-varying effective bias $B_{\text{eff},t} = \sum_m c_{t,m} B_m$. An oracle state decoder (exact Poisson log-likelihood) is also computed and saved as an upper bound for any inference algorithm.

Contents

1 Overview

The pipeline produces ground-truth data for a network of N neurons divided into N_E excitatory and $N_I = N - N_E$ inhibitory source cells. The network obeys Dale's principle in the natural matrix convention used by the simulator: each source neuron has outgoing off-diagonal weights of one sign only. All neurons share one connectivity matrix A_{true}; what differs across M states is only the bias (log-baseline firing-rate) vector B_m.

Stage 1 (Section 3). `gen_daleMatrices3c.py` builds and saves:

- a sparse binary connectivity mask $E_{\text{true}} \in \{0,1\}^{N\times N}$;
- a single weight matrix A_{true} with $\rho(A_{\text{true}}) = R$;
- M bias vectors $B_{\text{true}} \in \mathbb{R}^{M\times N}$, one per state.

Stationary spikes under each B_m are simulated and their firing-rate statistics are reported.

Stage 2 (Section 4). `gen_nonStationarySpikes3c.py` loads the Stage-1 output and:

- draws a target-state sequence $S_{\text{true}}[t]$ via a Markov chain or round-robin schedule;
- maintains a simplex coefficient vector $c_t \in \Delta^{M-1}$ that blends the M bias vectors smoothly;
- samples $Y_t \sim$ Poisson from the resulting time-varying Poisson GLM;
- computes the oracle state $S_{\text{oracle}}[t]$ by maximum Poisson log-likelihood.

Notation. N: total neurons; N_E: excitatory count (default $\lfloor 0.8N \rfloor$); $N_I = N - N_E$: inhibitory count. M: number of states (= `len(Boffsets)`). Δt: time-bin width in seconds (default 0.01 s). T: number of time bins. R: target spectral radius (default 0.3). $\eta_{\text{clip}} = 5$: overflow-prevention threshold ($\approx [10^{-3}, 10^{3}]$ Hz effective rate range). Matrix convention in the simulator: $(A_{\text{true}})_{ij}$ is the lag-1 effect of source neuron j on target neuron i, because the code evaluates $\eta_t = A_{\text{true}} Y_{t-1} + B_t$. Therefore source neuron j's outgoing weights are column j, and Dale's law is a column-wise off-diagonal sign constraint. The diagonal A_{jj} is a self-history term; it is forced non-positive for stability and is excluded from the source-type interpretation.

2 Input and Output File Pattern

Stage 1 input and output. `gen_daleMatrices3c.py` takes no input files. The root directory

`<basePath>`

and the output subdirectory

`<basePath>/truthDale`

must already exist; the program asserts both paths and does not create `truthDale`. If `--dataName` is not provided, the output stem is generated as

`daleN<num_neurons>_<6hex>`.

The two Stage 1 files are

`<basePath>/truthDale/<dataName>.simTruth.npz`

and

<basePath>/truthDale/<dataName>.spikes.npz.

The .simTruth.npz file is the required input for Stage 2. The .spikes.npz file in truthDale contains stationary sanity-check spikes, not the final switching dataset.

Stage 2 input and output. gen_nonStationarySpikes3c.py reads

<basePath>/truthDale/<inputStates>.simTruth.npz.

It creates <basePath>/spikesData if needed. If --dataName is not provided, the output stem is

<inputStates>_<6hex>.

The two Stage 2 files are

<basePath>/spikesData/<dataName>.spikes.npz

and

<basePath>/spikesData/<dataName>.prismTruth.npz.

The spikesData/<dataName>.spikes.npz file is the input consumed by prism_EM_train3c.py and by the EM-FDR-bagging driver.

3 Stage 1: Dale Matrix Generation

3.1 Command-Line Parameters

CLI flag	Default	Description
--num_neurons N	50	Total neuron count
--num_excite N_E	$\lfloor 0.8N \rfloor$	Excitatory neuron count
--edge_prob $[p_\ell, p_h]$	[0.05, 0.20]	Per-neuron connectivity probability range
--spectral_radius R	0.3	Target spectral radius
--idleRate $[f_{\min}, f_{\max}]$	[15, 30] Hz	Idle firing-rate band
--Boffsets $\{\delta_m\}$	[0.0]	Per-state log-rate offsets (one bias vector per entry)
--num_steps T	10 001	Steps for the stationary sanity-check simulation
--step_size Δt	0.01 s	Time-bin width
--basePath	*(scratch dir)*	Root output directory

Table 1: Key command-line arguments of gen_daleMatrices3.py.

3.2 Algorithm

Algorithm 1 Dale Matrix Generation (gen_daleMatrices3c.py)

1: **Step 1:** Generate sparse binary connectivity mask $E_{\text{true}} \in \{0,1\}^{N\times N}$ (generate_sparse_mask)
2: **Step 2:** Generate weight matrix A_{true} with Dale E/I balance, negative self-loops, and spectral radius R (init_W)
3: **Step 3:** For each offset δ_m in Boffsets:
4: Generate bias vector B_m (set_flat_selfSpiking)
5: Simulate stationary spikes $Y^{(m)}$ (gen_stationary_lag1_poisson)
6: Compute firing-rate statistics (estimate_rates)
7: **Step 4:** Save ground-truth and spike files

3.2.1 Step 1: Sparse Connectivity Mask

Each neuron i independently draws its connection probability:

$$q_i \sim \text{Uniform}(p_\ell,\ p_h). \tag{1}$$

The binary mask entry is

$$(E_{\text{true}})_{ij} = \begin{cases} 1 & \text{if } u_{ij} < q_i, \quad u_{ij} \sim \text{Uniform}(0,1) \text{ i.i.d.}, \\ 0 & \text{otherwise.} \end{cases} \tag{2}$$

Self-loops are *always* included: $(E_{\text{true}})_{ii} = 1$ for all i, encoding the mandatory negative self-connection added below.

3.2.2 Step 2: Weight Matrix with Dale's Principle

E/I balance ratio. Let $r = N_E/N_I$. This ratio is used to set the mean inhibitory weight magnitude so that the expected signed row-sum of the full matrix is approximately zero.

Raw weight assignment (source-column based, weight spread $v = 0.2$).

$$W_{ij} = \begin{cases} \text{Uniform}(1-v,\ 1+v) & j \in \{0, \ldots, N_E - 1\} \quad \text{(excitatory source columns)}, \\ \text{Uniform}(-r(1+v),\ -r(1-v)) & j \in \{N_E, \ldots, N-1\} \quad \text{(inhibitory source columns)}. \end{cases} \tag{3}$$

Because the *column* index j determines the source neuron's type, every nonzero off-diagonal outgoing weight from an excitatory source is positive and every nonzero off-diagonal outgoing weight from an inhibitory source is negative before the special diagonal rule below. This matches the model equation directly: no transposition is needed anywhere in the pipeline.

Applying the connectivity mask.

$$W \leftarrow W \odot E_{\text{true}}. \tag{4}$$

Enforcing negative self-loops. Any positive diagonal entry (excitatory self-loop) is sign-flipped:

$$W_{ii} \leftarrow -|W_{ii}| \quad \forall i, \tag{5}$$

so that all self-connections are inhibitory (stabilising).

Spectral normalization. Let $\rho_0 = \max_k |\lambda_k(W)|$ be the current spectral radius. The connectivity matrix is set to

$$A_{\text{true}} \leftarrow \frac{R}{\rho_0} W, \tag{6}$$

so $\rho(A_{\text{true}}) = R$ exactly. Rescaling preserves all signs, the zero pattern, Dale's law, and the negative self-loops. For $R < 1$ the operator A_{true} is contracting, which is a sufficient condition for the stationary Poisson VAR(1) to have a well-defined stationary distribution.

3.2.3 Step 3: Bias Vectors and Stationary Spike Check

Bias generation (`set_flat_selfSpiking`). A size-scaling factor $s = \sqrt{N/100}$ adjusts the excitatory correction for network size. For each state offset δ_m (index $m = 0, \ldots, M-1$), independent draws

$$B_m^{(i)} \sim \text{Uniform}\big(\ln(R\, f_{\min}),\ \ln(R\, f_{\max})\big), \quad i = 0, \ldots, N-1, \tag{7}$$

followed by state- and type-dependent shifts:

$$B_m^{(i)} \leftarrow \begin{cases} B_m^{(i)} + 1 + \delta_m - 1.5R - s & i < N_E \quad \text{(excitatory)}, \\ B_m^{(i)} + \delta_m & i \geq N_E \quad \text{(inhibitory)}. \end{cases} \tag{8}$$

The excitatory correction $-(1.5R+s-1)$ lowers the baseline excitatory rate relative to inhibitory neurons, preventing runaway excitation. Larger δ_m uniformly raises activity across the network, creating distinct mean-rate states.

Stationary spike simulation (evaluation only). For each bias vector B_m a lag-1 Poisson VAR(1) process is simulated over T bins:

$$\eta_t = A_{\text{true}}\, Y_{t-1} + B_m, \qquad \tilde{\eta}_t = \min(\eta_t,\ \eta_{\text{clip}}), \qquad \lambda_t = \exp(\tilde{\eta}_t)\, \Delta t, \qquad Y_t \sim \text{Poisson}(\lambda_t), \tag{9}$$

with initial condition $Y_0 \sim \text{Poisson}(\exp(B_m)\, \Delta t)$. These stationary spike arrays (shape (T, N), one per bias vector) are used *only* to verify that the firing rates are in a plausible range; they are *not* the final non-stationary output.

Firing-rate statistics computed via `estimate_rates` (`UtilDalePoisson.py`) include:

- mean per-neuron firing rates (Hz),
- rate variance over non-overlapping $T_{\text{var}} = 5\,\text{s}$ windows (Hz^2),
- Fano factors (Var[count]/Mean[count]),
- consecutive cross-neuron coincidence rate (Hz per neuron).

3.3 Output Files (Stage 1)

All files are written to `<basePath>/truthDale/`.

File	Array key	Shape	Description
`<dataName>.simTruth.npz`	`A_true`	(N, N) float	Shared weight matrix, $\rho(A_{\text{true}}) = R$
	`B_true`	(M, N) float	Per-state bias vectors
	`E_true`	(N, N) int	Binary connectivity mask
`<dataName>.spikes.npz`	`spikes`	(M, T, N) uint8	Stationary spikes per state (sanity check)
	`single_rates`	(M, N) float	Mean firing rates (Hz) per state
	`sigle_rates_var`	(M, N) float	Rate variance (Hz^2) per state
	`single_fano_fact`	(M, N) float	Fano factors per state

Table 2: Output arrays of Stage 1. $M =$ `len(Boffsets)`; $T =$ `num_steps`.

The metadata dictionary `dale_conf` records N, N_E, R, edge-probability range, idle-rate range, and offset list. The metadata dictionary `evol_conf` records T, Δt, total simulated time, and η_{clip}.

4 Stage 2: Non-Stationary Spike Generation

4.1 Command-Line Parameters

CLI flag	Default	Description
`--inputStates`	—	Base name of the `.simTruth.npz` file (required)
`--num_steps` T	from `evol_conf`	Number of time bins
`--true_dwell_sec` t_d	1.0 s	Mean dwell time per state
`--state_change_speed` ν	0.33	Max simplex-coefficient change per bin
`--schedule`	`mc`	State schedule: Markov chain (`mc`) or round-robin (`rr`)
`--seed`	42	Random seed

Table 3: Key command-line arguments of `gen_nonStationarySpikes3.py`.

4.2 Algorithm

Algorithm 2 Non-Stationary Spike Generation (`gen_nonStationarySpikes3.py`)

1: Load $(A_{\text{true}}, B_{\text{true}})$ from `<inputStates>.simTruth.npz`
2: Build target-state sequence $S_{\text{true}}[t]$ via selected schedule (Section 4.3)
3: Simulate switching spikes and record simplex coefficients C_{true} (`simulate_switching_poisson`, Section 4.4)
4: Compute oracle state sequence S_{oracle} (`compute_oracle_states_comA`, Section 4.5)
5: Compute firing-rate statistics (`estimate_rates`)
6: Save spike and ground-truth files

4.3 State Scheduling

The mean dwell time in bins is $\tau_d = \lceil t_d/\Delta t \rceil$.

Markov chain (`mc`, default). A discrete-time Markov chain with geometric dwell times. The exit probability per bin is $p_{\text{exit}} = 1/\tau_d$; on exit the chain transitions uniformly to one of the $M-1$ other states. The symmetric $M \times M$ transition matrix stored in the output is:

$$T_{sm} = \begin{cases} 1 - p_{\text{exit}} & s = m \quad \text{(self-transition)}, \\ p_{\text{exit}}/(M-1) & s \neq m \quad \text{(cross-transition)}. \end{cases} \tag{10}$$

A minimum dwell of $\tau_{\min} = \lceil 0.3\,\tau_d \rceil$ bins is enforced to prevent spuriously short visits. State coverage across the full record is uncontrolled and subject to random fluctuations.

Round-robin (`rr`). States cycle deterministically as $0, 1, \ldots, M-1, 0, 1, \ldots$ Each visit lasts exactly τ_d bins (the last visit in the record is trimmed to fit T). This guarantees that all states receive equal coverage ($T/M \pm \tau_d$ bins each), eliminating sampling variability.

4.4 Smooth Switching Poisson Simulation

Simplex coefficient vector. A vector $c_t \in \Delta^{M-1}$ (non-negative entries summing to 1) tracks the current mixture of states. The one-hot target for the scheduled state at bin t is $e_m = \mathbf{1}[m = S_{\text{true}}[t]]$. At

each bin, c_t is updated by a clipped gradient step and renormalized:

$$c_t \leftarrow c_{t-1} + \mathrm{clip}\big(e - c_{t-1},\ -\nu,\ \nu\big), \tag{11}$$

$$c_t \leftarrow c_t \,/\, \|c_t\|_1. \tag{12}$$

The speed parameter $\nu \in (0, 1]$ controls how quickly the mixture tracks the target: $\nu = 1$ gives instantaneous switching; small ν produces gradual multi-bin transitions. The transition duration is approximately $1/\nu$ bins.

Effective bias. The convex mixture of per-state bias vectors is

$$B_{\mathrm{eff},t} = \sum_{m=0}^{M-1} c_{t,m}\, B_m = \sum_{m=0}^{M-1} c_{t,m}\, B_{\mathrm{true}}[m, :]. \tag{13}$$

This interpolates smoothly between the per-state firing-rate baselines as the state label switches.

Non-stationary spike generation. At every bin $t = 0, \ldots, T-1$:

$$\eta_t = A_{\mathrm{true}}\, Y_{t-1} + B_{\mathrm{eff},t}, \qquad \tilde{\eta}_t = \min(\eta_t,\ \eta_{\mathrm{clip}}), \qquad \lambda_t = \exp(\tilde{\eta}_t)\,\Delta t, \qquad Y_t \sim \mathrm{Poisson}(\lambda_t). \tag{14}$$

This is the same Poisson VAR(1) model as in Eq. (9), except that the bias $B_{\mathrm{eff},t}$ is now time-varying. The connectivity matrix A_{true} is unchanged across all states.

4.5 Oracle State Decoder

An oracle decoder assigns the most likely state to each bin using the *exact* ground-truth parameters $(A_{\mathrm{true}}, \{B_m\})$:

$$S_{\mathrm{oracle}}[t] = \arg \max_{m \in \{0,\ldots,M-1\}} \sum_{j=0}^{N-1} \Big[Y_{t,j}\, \tilde{\eta}_{t,j}^{(m)} - \exp\big(\tilde{\eta}_{t,j}^{(m)}\big)\, \Delta t \Big], \tag{15}$$

where $\eta_{t,j}^{(m)} = \big(A_{\mathrm{true}}\, Y_{t-1}\big)_j + B_{m,j}$ and $\tilde{\eta}^{(m)} = \min(\eta^{(m)}, \eta_{\mathrm{clip}})$. This is the exact Poisson log-likelihood score for state m given the observed spike vector Y_t, computed independently at each bin. The oracle accuracy is an upper bound for any latent-state inference algorithm.

Per-state oracle scores and overall accuracy are printed at the end of the run and stored in the `oracle_eval` sub-dictionary of the metadata.

4.6 Output Files (Stage 2)

All files are written to `<basePath>/spikesData/`.

The metadata dictionary `evol_conf` records T, Δt, total time, ν, requested and effective dwell times (seconds and bins), M, random seed, and schedule type. The `oracle_eval` sub-dictionary records the overall oracle accuracy and per-state breakdown.

5 Notation Summary

6 Full Pipeline and Suggested Commands

1. **Generate ground-truth matrices** (Stage 1):

File	Array key	Shape	Description
`<dataName>.spikes.npz`	`spikes`	(T, N) int32	Non-stationary spike counts
	`single_rates`	$(N,)$ float	Mean firing rates over the full record (Hz)
`<dataName>.prismTruth.npz`	`A_true`	(N, N) float	Shared weight matrix (copy from Stage 1)
	`B_true`	(M, N) float	Per-state bias vectors (copy from Stage 1)
	`S_true`	$(T,)$ int32	Target state index at each bin
	`C_true`	(T, M) float32	Simplex coefficient vector at each bin
	`S_oracle`	$(T,)$ int32	Oracle (max-likelihood) state at each bin
	`state_transition`	(M, M) float32	Transition matrix used/implied by schedule
	`sigle_rates_var`	$(N,)$ float	Per-neuron rate variance (Hz^2)
	`single_fano_fact`	$(N,)$ float	Per-neuron Fano factors

Table 4: Output arrays of Stage 2. $T =$ `num_steps`; $M =$ `num_states`.

Symbol	Domain	Description
N	$\mathbb{Z}_{>0}$	Total number of neurons
N_E	$\mathbb{Z}_{>0}$	Excitatory neuron count (rows $0 \dots N_E - 1$)
$N_I = N - N_E$	$\mathbb{Z}_{>0}$	Inhibitory neuron count
M	$\mathbb{Z}_{>0}$	Number of states (= `len(Boffsets)`)
T	$\mathbb{Z}_{>0}$	Number of time bins
Δt	$\mathbb{R}_{>0}$	Time-bin width (s); default 0.01
R	$(0, 1)$	Target spectral radius
$r = N_E/N_I$	$\mathbb{R}_{>0}$	E/I balance ratio
$v = 0.2$	—	Weight spread (hard-coded)
E_{true}	$\{0, 1\}^{N \times N}$	Binary connectivity mask
A_{true}	$\mathbb{R}^{N \times N}$	Shared weight matrix, $\rho(A_{\text{true}}) = R$
B_m	$\mathbb{R}^N$	Bias (log-baseline rate) vector for state m
B_{true}	$\mathbb{R}^{M \times N}$	Stacked per-state biases
Y_t	$\mathbb{Z}_{\geq 0}^N$	Spike-count vector at bin t
η_{clip}	$\mathbb{R}_{>0}$	Rate clipping threshold; fixed at 5
$S_{\text{true}}[t]$	$\{0, \dots, M-1\}$	Scheduled target state at bin t
c_t	Δ^{M-1}	Simplex coefficient vector at bin t
ν	$(0, 1]$	Max coefficient change per bin (`state_change_speed`)
$B_{\text{eff},t}$	$\mathbb{R}^N$	Effective bias: convex mixture $\sum_m c_{t,m} B_m$
τ_d	$\mathbb{Z}_{>0}$	Mean dwell time in bins
$S_{\text{oracle}}[t]$	$\{0, \dots, M-1\}$	Oracle max-likelihood state at bin t

Table 5: Notation used throughout this document.

```
mkdir -p $basePath/truthDale
./gen_daleMatrices3c.py --num_neurons 100 --num_excite 70 \
    --spectral_radius 0.5 --Boffsets 0 10 20 --dataName myData \
    --basePath $basePath
```

Produces `myData.simTruth.npz` (A_{true}, B_{true}, E_{true}; $M = 3$ states here) and `myData.spikes.npz` (stationary sanity-check arrays, shape $(3, T, N)$), both under `$basePath/truthDale/`.

2. **Generate non-stationary spike train** (Stage 2):

```
./gen_nonStationarySpikes3c.py --basePath $basePath \
    --inputStates myData --dataName myData_sw \
    --true_dwell_sec 1.0 --schedule mc
```

Produces `myData_sw.spikes.npz` (shape (T, N)) and `myData_sw.prismTruth.npz` (S_{true}, C_{true}, S_{oracle}, transition matrix, and copies of A_{true}, B_{true}), both under `$basePath/spikesData/`.

3. **Fit with PRISM-EM**:

```
torchrun --standalone --nnodes=1 --nproc_per_node=4 \
  ./prism_EM_train3c.py \
    --basePath $basePath --dataName myData_sw \
    --num_states 3 --num_em_iters 4 --m_epochs 16 \
    --time_range_sec 0 80
```

Writes `$basePath/prismFit/<fitName>.prismEM.npz`, where `<fitName>` is either supplied by `--fitName` or generated as `myData_sw-EM-<6hex>`.

7 Key Design Choices

Shared A_{true} across all states. The synaptic weight matrix is identical in every state; only the bias vector B_m differs between states. This models a circuit whose connectivity is fixed but whose input drive (e.g. neuromodulatory tone, sensory stimulus level) shifts the population-wide activity baseline between states. The inference task is therefore to recover both A_{true} and $\{B_m\}$ from a single non-stationary spike record.

Source-column Dale convention. The simulator uses A_{ij} as source j into target i. Therefore column j contains source neuron j's outgoing effects. The generator assigns signs by column: columns $0 \dots N_E - 1$ have positive off-diagonal entries and columns $N_E \dots N - 1$ have negative off-diagonal entries. Diagonal entries are forced non-positive as stabilizing self-history terms and are not part of the excitatory/inhibitory source classification. This is the same convention used by PRISM-EM and by the EM-FDR aggregate when they infer source neuron type from fitted columns.

Spectral radius and stability. $\rho(A_{\text{true}}) = R < 1$ is a sufficient condition for the stationary Poisson VAR(1) to admit a well-defined stationary distribution. In the non-stationary regime, stability is maintained instantaneously because A_{true} is unchanged; only the bias shifts.

Smooth simplex interpolation. The clipped gradient update (Eq. (12)) prevents abrupt discontinuities in $B_{\text{eff},t}$ at state-boundary bins. This mimics a biological circuit that cannot switch network state instantaneously. The mixing speed ν is independent of the mean dwell time and can be set to match a desired biophysical transition timescale.

Oracle score as a decoder upper bound. The oracle uses exact ground-truth parameters and per-bin maximum-likelihood, achieving the best possible state assignment from the spike observations. Any EM or variational algorithm must approach, but cannot exceed, this score. The oracle score therefore quantifies how much information about the latent state is present in the spikes at all, distinguishing fundamental ambiguity from algorithm suboptimality.